Lecture 5: Parallelism

Building Large Foundation Models from Scratch · WS 2026/27

Asst. Prof. Dr. Martin Pawelczyk

Faculty of Computer Science, University of Vienna

Thu 15 Oct 2026

Where we are

Admin, 2 minutes

Today

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

One GPU is not enough

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

Three reasons illustrated using Llama 3 405B

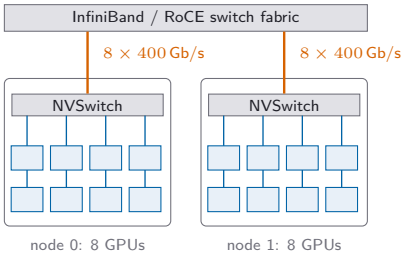
	the number	one H100	consequence
memory	$405\text{B} \times 16$ bytes $= 6.5$ TB of training state	80 GB	≥ 81 GPUs just to hold it
time	$6 \cdot 405\text{B} \cdot 15.6\text{T} = 3.8 \cdot 10^{25}$ FLOPs	400 TFLOP/s achieved	3000 years; 67 days on 16,384
batch	16M tokens per step $= 2048$ sequences of 8k	~ 1 sequence per step	the batch spans ~ 2000 GPUs

- Three different constraints: sharding memory, splitting work and splitting the batch are different operations, each with communication of a different shape.
- The 16,384 GPUs reached 38–43 % MFU [llama3]. The rest is communication that could not be hidden, plus Lecture 4's GPU-level losses.

Key idea

Lecture 4, one level up: the slowest link sets the pace. Move as few bytes as possible over it, overlap the rest with compute.

The bandwidth ladder: what a byte costs at each level



blue: NVLink inside a node, 450 GB/s per GPU and direction
orange: InfiniBand or RoCE between nodes, 50 GB/s per GPU

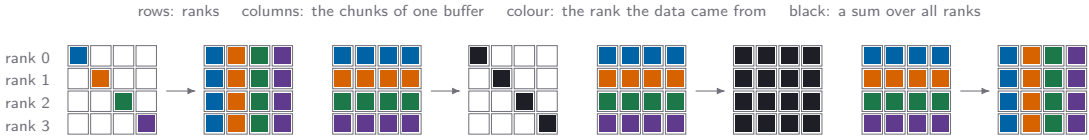
link	scope	per GPU	vs. HBM
SRAM	inside the SM	~ 30 TB/s	$10\times$
HBM3	one GPU	3.35 TB/s	1
NVLink (NVSwitch)	8-GPU node	450 GB/s	$1/7$
InfiniBand / RoCE	whole cluster	50 GB/s	$1/67$
PCIe 5 to the host	one GPU	64 GB/s	$1/50$

- Latency: ~ 10–$20\,\mu$s to start a collective, more across nodes. Small messages are latency-bound; bytes matter once they are MBs.
- The traffic that cannot be avoided goes on NVLink; the inter-node links get what is rare or overlappable.

Collectives and their costs

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

Four collectives are enough

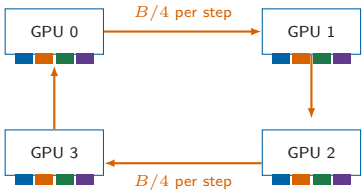


- **all-reduce** = **reduce-scatter** + **all-gather**. ZeRO and FSDP are built on taking the two halves apart.
- Also needed: point-to-point `send`/`recv` (pipeline stages) and a `broadcast` of the initial weights.

Takeaway

Bytes per rank for a buffer of B bytes on n ranks: $\frac{n-1}{n}B$ for all-gather, reduce-scatter and all-to-all; $2\frac{n-1}{n}B$ for all-reduce.

Ring all-reduce: optimal in bytes, paid in latency



each GPU holds B bytes in $n = 4$ chunks; every step it sends one chunk to the right and adds the one arriving from the left

- **Reduce-scatter**: $n-1$ steps of B/n bytes, after which GPU i owns the finished chunk i. **All-gather**: $n-1$ more steps pass the finished chunks around.
- Per GPU $2\frac{n-1}{n}B$ bytes in $2(n-1)$ steps: $T \approx 2(n-1)\,\alpha + 2\frac{n-1}{n}\,B/\beta$ (α: latency per step, β: link bandwidth). Large B: $2B/\beta$, *independent of* n; small B: $2(n-1)\alpha$.

Takeaway

Send few, large messages: DDP buckets gradients into 25 MB pieces, FSDP works per Transformer block. We fit α and β in the demo.

The API: `torch.distributed` in nine lines

```python
import torch, torch.distributed as dist
dist.init_process_group("nccl")   # "gloo" on CPUs
rank, n = dist.get_rank(), dist.get_world_size()
g = torch.randn(10**6, device=f"cuda:{rank}")
dist.all_reduce(g, op=dist.ReduceOp.SUM)  # in place
shard = torch.empty(g.numel() // n, device=g.device)
dist.reduce_scatter_tensor(shard, g)  # chunk `rank`
dist.all_gather_into_tensor(g, shard) # = all_reduce
if rank == 0: dist.send(g, dst=1)     # pipeline edge
```

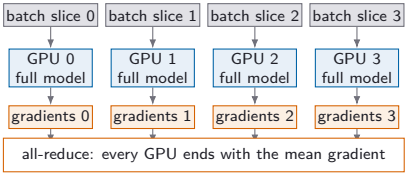
`torchrun --nproc_per_node=8 train.py`: one process per GPU, with `RANK` and `WORLD_SIZE` set.

- One process per GPU, all running the same script. Collectives are **contracts**: every rank calls the same collective in the same order, or everyone hangs.
- **Process groups** carve the world into dimensions: `init_device_mesh("cuda", (16, 8), ("dp", "tp"))` gives each rank a DP group and a TP group.

Data parallelism: DDP, ZeRO, FSDP

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

Data parallelism: replicate the model, split the batch

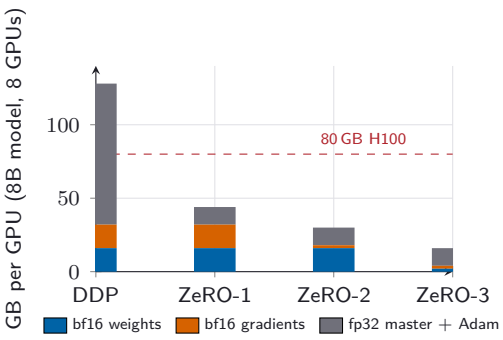


Takeaway

Gradient accumulation is data parallelism in time: what you do on Colab.

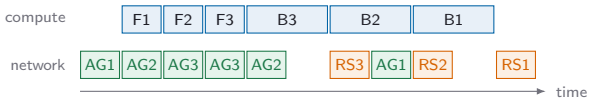
- Each GPU: its slice of the batch, a full forward and backward pass, one all-reduce of the gradients. Mathematically one large batch.
- **DDP** overlaps: gradient buckets are all-reduced while the backward pass is still running, last layers first [li2020].
- Cost per step: $2\frac{n-1}{n}$ of the gradient bytes, whatever the batch; a larger per-GPU batch amortises it.
- Limit: every GPU holds all 16 bytes per parameter. On 80 GB that is a 5B model before activations.

ZeRO: the state is replicated n times for no reason



- 12 of the 16 bytes per parameter are optimizer state, touched once per step. **ZeRO-1** shards it: each GPU updates $1/n$ of the parameters; an all-gather spreads the result.
- **ZeRO-2** also shards the gradients: the all-reduce becomes a reduce-scatter; same bytes as DDP.
- **ZeRO-3** also shards the weights, all-gathered before use in the forward and again in the backward pass: $1.5\times$ DDP's traffic, memory $\propto 1/n$. This is **FSDP**; per GPU 128, 44, 30, 16 GB.

FSDP: ZeRO-3 as it runs



three blocks, one micro-batch. AGk: all-gather the weights of block k; RSk: reduce-scatter its gradients. Only the last reduce-scatter is exposed.

```
from torch.distributed.fsdp import fully_shard
for block in model.blocks:        # one unit per block
    fully_shard(block, mesh=mesh["dp"])
fully_shard(model, mesh=mesh["dp"])
```

- The unit is a Transformer block: its shards are all-gathered just before use and freed right after; the next block's all-gather is **prefetched** during this block's compute.
- After a block's backward pass its gradients are reduce-scattered; the optimizer only ever sees shards.
- Mixed precision: bf16 copies for compute, fp32 shards as master weights; CPU offload if even $1/n$ is too much [zhao2023].

A roofline for communication: when overlap works

- Communication hides behind compute only if it is *shorter*. Per GPU and step, T_{GPU} tokens, b bytes per parameter, GPU rate F, link rate β:

$$\frac{6\,P\,T_{\text{GPU}}}{F} \geq \frac{b\,P}{\beta} \quad \Longleftrightarrow \quad T_{\text{GPU}} \geq \frac{b}{6} \cdot \frac{F}{\beta}.$$

- F/β is the **ridge point of the link**: H100 over NVLink $989\text{T}/450\text{G} \approx 2200$ FLOP per byte; over InfiniBand $\approx 19{,}800$.

	b	NVLink	InfiniBand
DDP, bf16 gradients	4	$T_{\text{GPU}} \geq 1.5\text{k}$	$\geq 13\text{k}$
FSDP (AG, AG, RS)	6	$\geq 2.2\text{k}$	$\geq 20\text{k}$

- 20k tokens are 2.5 sequences of 8k per GPU per step. Llama 3's 16M tokens over 16,384 GPUs are 1k per GPU: FSDP alone, over the inter-node links, could not hide its traffic.
- At 40 % MFU compute is slower and the thresholds drop by $2.5\times$.

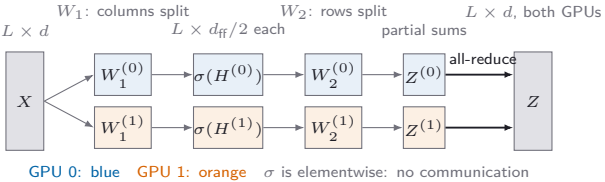
Key idea

A ridge point belongs to a pair (compute, link). A layout puts each traffic on a link whose ridge it clears.

Tensor, pipeline, sequence, expert parallelism

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

Tensor parallelism: split every matmul across GPUs



Attention: heads are the split unit. Each GPU holds n_{heads}/n heads of W_Q, W_K, W_V (columns) and the matching rows of W_O; one all-reduce after W_O.

- Megatron's trick [shoeybi2019]: split W_1 by columns, so each GPU computes half the hidden units and applies σ locally; split W_2 by rows, so each GPU holds a partial sum. One all-reduce gives everyone Z.
- Per block: two all-reduces in the forward pass (MLP, attention), two in the backward pass.
- Weights, gradients and optimizer state are sharded n ways for free; X and Z are replicated, the wide intermediates are not.

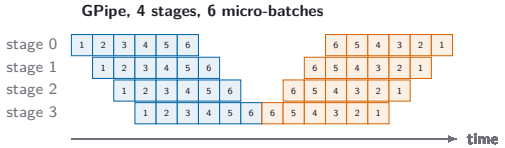
What tensor parallelism costs, and why it stays inside the node

- Per token and block each GPU moves $4 \times 2\frac{n-1}{n} \times 2d \approx 16d$ bytes and computes $6 \cdot 12d^2/n$ FLOPs, its share of $12d^2$ parameters: $\dfrac{72d^2/n}{16d} = \dfrac{4.5\,d}{n}$ FLOP per byte, against the link's ridge.

model	d	TP 2	TP 4	TP 8
Llama 3 8B	4096	9.2k	4.6k	2.3k
Llama 3 70B	8192	18k	9.2k	4.6k
Llama 3 405B	16384	37k	18k	9.2k
NVLink ridge			2.2k	
InfiniBand ridge			19.8k	

- Every entry clears NVLink, barely for the 8B at TP 8; almost none clears InfiniBand: **TP degree $\leq$ GPUs per node**.
- **Sequence parallelism** [korthikanti2022]: shard the norms between the matmuls along the sequence; each all-reduce becomes reduce-scatter + all-gather. Same bytes, activations $\div n$.

Pipeline parallelism: split the layers into stages



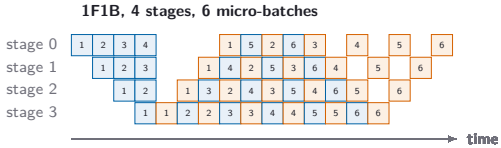
blue: forward of micro-batch i; orange: its backward (drawn equally long); white: idle, the **bubble**.

Takeaway

Cheap in bytes; paid in idle time and in memory imbalance between stages.

- Stage = a run of consecutive blocks on one GPU. Only boundary activations move, point to point: $2Ld$ per micro-batch, 268 MB for 8k tokens at $d = 16{,}384$: **5 ms** on InfiniBand against a 0.4 s stage.
- The price is idle time: p stages and m micro-batches leave a bubble of $\frac{p-1}{m}$ of the useful time: here $\frac{3}{6}$, with $m = 64$ only 5 %.
- So pipelines want many micro-batches per step; stage 0 holds up to m of them until their backward pass.

Pipeline schedules: less memory first, then less bubble



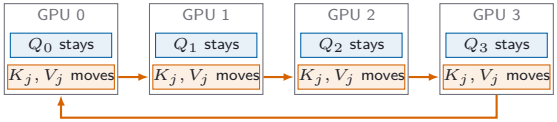
the same 18 idle cells as GPipe, but a stage never holds more than p micro-batches of activations: each backward pass starts as soon as it can.

schedule	bubble	idea
GPipe (2019)	$\frac{p-1}{m}$	all forwards, then all backwards
1F1B (2019)	$\frac{p-1}{m}$	alternate; memory $\propto p$, not m
interleaved (2021)	$\frac{p-1}{v\,m}$	v stage chunks per GPU; $v\times$ the messages
zero bubble (2024)	$\to 0$	split the backward pass into input- and weight-gradient halves; fill bubbles with the latter
DualPipe (2024)	small	micro-batches from both ends; overlaps the MoE all-to-all

Takeaway

Pipeline research is bubble accounting; the weight-gradient pass is the slack everyone now uses.

Context parallelism: when one sequence does not fit



ring attention: each round, GPU i scores Q_i against the K, V block that just arrived, folds the result in with the online softmax, and forwards the block. After n rounds every query has seen every key.

- At 128k tokens, *one* sequence's saved activations at $d = 16{,}384$ are 4 GB per tensor and layer: split the sequence across GPUs.
- Everything but attention is per token. Attention passes K, V blocks around a ring [liu2023ring], each transfer overlapped with compute; or all-to-all over heads (Ulysses) [jacobs2023].
- Causal masking gives late chunks more work: assign chunks zig-zag. Llama 3: CP 16 at 128k with a plain all-gather of K, V, 4 KB per token under GQA [llama3].

Expert parallelism: experts live on different GPUs



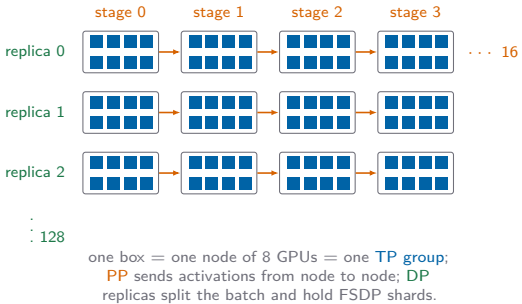
cells: tokens, coloured by the expert the router chose (one expert per GPU here). Dispatch and combine are all-to-alls; in between, each GPU runs its expert on whatever arrived.

- Experts are independent MLPs: place E/n per GPU [lepikhin2020]. A token travels to its k experts and back, $2 \cdot k \cdot 2d$ bytes: **229 KB** per token for DeepSeek-V3 ($k = 8$, $d = 7168$).
- Expert compute per token, $6 \cdot k \cdot 3d\, d_{\mathrm{e}} = 2.1$ GFLOP, over 229 KB: ≈ 9k FLOP per byte, below the inter-node ridge of 20k. And the traffic is uneven: a popular expert's GPU is everyone's straggler.
- DeepSeek-V3's answers [deepseekv3]: EP 64 over 8 nodes; **node-limited routing**, a token's experts in at most 4 nodes; fp8 dispatch; all-to-all kernels on 20 reserved SMs; DualPipe overlaps the rest with the other micro-batch's compute.

Putting it together

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

Combining them: the 4D layout of Llama 3 405B



GPUs	TP	CP	PP	DP	context	MFU
8192	8	1	16	64	8k	43 %
16384	8	1	16	128	8k	41 %
16384	8	16	16	8	128k	38 %

- Order [TP, CP, PP, DP], innermost first: the heaviest traffic gets the fastest link; TP never leaves the node, DP spans the cluster.
- 16M tokens per step in every row; the 128k stage trades DP for CP. The batch itself is set by the optimiser (Lecture 6).

Table 5 of the Llama 3 paper [llama3].

DeepSeek-V3: a different answer to the same constraints

hardware	2048 H800: H100 compute, NVLink 400 GB/s, IB 50 GB/s
model	671B total, 37B active; MLA; 256 routed + 1 shared expert
parallelism	PP 16 (DualPipe) $\times$ EP 64 (8 nodes) $\times$ ZeRO-1 DP
tensor parallelism	none
precision	fp8 matmuls; bf16 and fp32 where it matters
cost	2.79M GPU-hours for 14.8T tokens

Key idea

Llama 3 chose a dense model and the biggest cluster; DeepSeek chose a sparse model designed around a smaller cluster's links. Same calculation, different inputs.

- No TP: with EP 64 and 16 stages each GPU holds a few experts and a slice of the attention stack, small enough without splitting a matmul.
- The traffic moved into the all-to-all, and the model was shaped around it: node-limited routing, MLA's small activations, fp8 messages.
- DualPipe: two micro-batch streams in opposite directions; compute overlaps the other stream's dispatch and combine. 20 of 132 SMs do only communication.

Choosing a layout: the cheat sheet

	splits	what moves	bytes per GPU	degree
data / FSDP	the batch	gradients; under FSDP also the weights	4–6 per parameter per step	8–1000s
tensor	every matmul	activations, 4 all-reduces per block	$16d$ per token per block	≤ 8
pipeline	the layers	boundary activations, point to point	$2Ld$ per micro-batch	4–16
context	the sequence	K, V blocks	$4\,d_{kv}$ per token per block	2–16
expert	the experts	tokens, 2 all-to-alls per MoE block	$4kd$ per token	8–64

- What each needs: DP/FSDP, tokens per GPU above the link's ridge; TP, NVLink ($4.5d/n > 2200$); PP, many micro-batches; CP, long context; EP, balanced routing and a fast all-to-all.
- Rules of thumb: TP up to the node; PP until the model fits and the bubble is acceptable; DP/FSDP for the rest; CP only for long context; EP if and only if the model is an MoE.

On Colab

One GPU: gradient accumulation replaces DP; activation checkpointing replaces memory you do not have ($+33\,\%$ FLOPs).

Wrap-up

1. One GPU is not enough
2. Collectives and their costs
3. Data parallelism: DDP, ZeRO, FSDP
4. Tensor, pipeline, sequence, expert parallelism
5. Putting it together
6. Wrap-up

Summary

1. Three constraints force many GPUs: state that does not fit, compute that takes years, a batch of thousands of sequences. Each is split differently and each split has its own traffic.
2. Four collectives suffice; the all-reduce is a reduce-scatter plus an all-gather and costs $2\frac{n-1}{n}B$ bytes per GPU, independent of n for large buffers.
3. Data parallelism replicates; ZeRO/FSDP shards the 16 bytes per parameter for $1.5\times$ the traffic. It hides behind compute only if the tokens per GPU clear the link's ridge point: $\sim$ 2k on NVLink, $\sim$ 20k on InfiniBand.
4. Tensor parallelism splits matmuls ($4.5d/n$ FLOP per byte: inside the node only); pipeline parallelism splits layers (cheap bytes, a bubble of $\frac{p-1}{m}$); context and expert parallelism split the sequence and the experts.
5. A layout puts each kind of traffic on a link whose ridge it clears: Llama 3's $[8, 1\text{–}16, 16, 128]$ at 38–43 % MFU; DeepSeek-V3's PP $\times$ EP $\times$ ZeRO-1 with no TP, designed together with the model.

Next: Lecture 6 – Scaling laws (Thu 22.10., SR 6)

Coming up

- Power laws in N, D and compute; Kaplan versus Chinchilla, and what changed between them
- Fitting a scaling law from a handful of small runs: the A2 workflow
- Batch size, learning rate, the critical batch size; μP and hyperparameter transfer

Deadline: A1 due Wednesday 21.10., 23:59, on Moodle. A2 goes out the same day.

Reading

- Hoffmann et al. (2022), sections 1–3 (Chinchilla); Kaplan et al. (2020), sections 1–3
- For today: the Hugging Face Ultra-Scale Playbook (2025), from "Data parallelism" to "5D parallelism in a nutshell"
- Optional: Austin et al. (2025), How to scale your model, chapters 3–5

References

- Grattafiori et al. (2024). The Llama 3 herd of models. arXiv:2407.21783.
- DeepSeek-AI (2024). DeepSeek-V3 technical report. arXiv:2412.19437.
- Rajbhandari, Rasley, Ruwase & He (2020). ZeRO: Memory optimizations toward training trillion parameter models. *SC*.
- Korthikanti et al. (2022). Reducing activation recomputation in large Transformer models. arXiv:2205.05198.
- Austin et al. (2025). How to scale your model: A systems view of LLMs on TPUs. `https://jax-ml.github.io/scaling-book/`.